THE NATIONAL UNIVERSITY OF LESOTHO

Faculty of Science and Technology — Department of Mathematics and Computer Science

CS3520 — Computer Organisation and Architecture I

PRACTICAL LAB HANDOUT

Lab No.	Lab 1
Lab Title	Assembly Programming I — From Algorithm to Machine Code and Back
Supports Week(s)	Weeks 1–3 — the compilation path, the RISC-V instruction set, instruction formats and encoding
Duration	2 hours — one scheduled lab session
Venue / Group(s)	ICT Lab — Group A (Wed 08:00–10:00) · Group B (Fri 08:00–10:00) · Group C (Fri 11:10–13:10)
Submission Platform	Thuto — CS3520 site, Lab 1 submission item
Submission Deadline	Before the start of the next lab session (see Thuto for the exact cut-off)

Weighting Completion and your submitted values count toward your continuous lab-assessment record.

LEARNING OUTCOMES

On completion of this lab, you should be able to:

– Model a small computational problem in C++ before implementing it, and use that model to check your assembly. (CLO2)

– Translate a working C++ program by hand into RISC-V assembly, and verify that both produce the same result. (CLO2)

– Decompose a 32-bit instruction into its fields, and identify its format, opcode and register operands. (CLO2)

– Reconstruct assembly from raw machine code — the reverse of assembly. (CLO2) – Read an unfamiliar assembly program, explain what it does, and extend it correctly. (CLO2)

BEFORE YOU START

– Lab 0 completed — VS Code, Ripes and the course repository working on your machine.

– A C++ compiler available: g++ on Linux/macOS, or MinGW/WSL on Windows.

– Review the Week 3 lecture on instruction formats. Bring the RISC-V reference card — you will need the opcode and funct field values throughout.

TOOLS & RESOURCES

Tool / Resource	Purpose
VS Code	Editing both the C++ model and the assembly source

g++	Compiling and running the C++ model to obtain the expected answer
Ripes	Assembling, running, and inspecting the generated machine code
Course repository	github.com/khobatha/CS3520 — example programs used in Steps 1 and 6
RISC-V reference card	Opcode, funct3 and funct7 values for every instruction you will meet

PROCEDURE

Step 1 — Update Your Copy of the Course Repository

```
cd CS3520
git pull origin main
```

Confirm you can see the folder examples/2026/hello-world-assembly/ — you will need it in Step 6.

Step 2 — Model the Problem in C++ First

Problem: three integers a, b and c are given. Compute

```
result = (a + b) * 8 - c
```

Write model.cpp that sets a = 7, b = 5, c = 20, computes the result, and prints it. Compile and run it:

```
g++ -Wall -std=c++17 model.cpp -o model
./model
```

> Record the answer your C++ model prints. This is your expected result — in Step 3 your assembly must produce exactly the same number. Modelling first means you are debugging one thing at a time: the logic in C++, then the translation in assembly.

Step 3 — Translate the Model into RISC-V Assembly

Write task.s implementing the same calculation. Requirements:

- Store a, b and c in the .data section using .word.
- Load them into registers with lw, and keep the result in a register.
- Use slli to multiply by 8 rather than a multiply instruction — shifting left by 3 is the same operation, and far cheaper.
- Print the result using the print-integer environment call, then exit cleanly.

Assemble and run it in Ripes. If the number does not match Step 2, step through instruction by instruction and watch the registers until you find where they diverge.

Step 4 — Read the Machine Code (Assembly → Machine)

With task.s assembled, Ripes shows the 32-bit encoding beside each instruction. Choose three of your own instructions — one R-type, one I-type, one load or store — and complete this table by hand, then check each row against what Ripes displays:

Instruction	Format	opcode	funct3 / funct7	rd, rs1, rs2	Hex

Step 5 — Work Backwards (Machine → Assembly)

Below are three RISC-V instructions given only as machine code. Decode each one by hand — split it into fields, identify the format from the opcode, then name the registers — and write out the assembly instruction it represents:

```
0x00A00293
0x406283B3
0x00329293
```

Check your answers by entering the three instructions into Ripes and confirming the encodings match.

> This is exactly what a disassembler and a debugger do. Being able to work in both directions is what separates understanding an ISA from memorising a table of mnemonics.

Step 6 — Analyse and Extend an Example from the Repository

Open examples/2026/hello-world-assembly/hello-world.s. Read it carefully, then:

– Explain, in three or four sentences, what the program does — covering the role of the .data section, what la places in a0, and why a7 is loaded twice with different values.

– Extend it: keep the original message, then print a second message of your own after it. – Extend it once more: compute a small arithmetic result in a register and print it as an integer after the messages.

> The environment-call numbers in this example are simulator conventions, not Linux system calls — the example's own README says so. Check the numbers against your simulator before assuming they transfer.

Step 7 — Stretch Task: A First Look at Control Flow (Optional)

We have not yet covered branches in class — that is Week 4. Attempt this only once Steps 1–6 are complete and working.

A branch compares two registers and, if the test passes, jumps to a label instead of continuing to the next instruction:

```
beq t0, t1, Label # go to Label if t0 == t1
bne t0, t1, Label # go to Label if t0 != t1
```

Modify your Step 3 program so that it prints one message if the result is positive and a different message if it is not. Then change the values of a, b and c so that each message is produced in turn, and confirm both paths work.

WHAT TO SUBMIT

Submit a single ZIP archive named Lab1_<YourStudentNumber>.zip containing:

– model.cpp — your C++ model from Step 2

– task.s — your assembly implementation from Step 3

– hello-world-extended.s — your extended example from Step 6

– answers.txt or answers.md — the completed table from Step 4, your three decoded instructions from Step 5, and your written explanation from Step 6

Upload to the Lab 1 submission item on Thuto before the deadline. If you attempted Step 7, include that file too and say so in your answers file.

GETTING HELP

– Lab assistants are on hand throughout the session — ask before searching online. –

Outside lab hours: post on the CS3520 course forum, or email ko.setetemela@nul.ls. –

Consultation hours are posted on the office notice board.